Cyberscope

A TAC Security Company

Audit Report

Robin Flow

July 2026

Network	Robinhood
Address	0x3068bF0522d721e674aB9A252fDC50bB614b3caE
Audited by	© cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	RobinFlowStreams
Compiler Version	v0.8.24+commit.e11b9ed9
Optimization	200 runs
Explorer	https://robinhoodchain.blockscout.com/address/0x3068bF0522d721e674aB9A252fDC50bB614b3caE
Address	0x3068bF0522d721e674aB9A252fDC50bB614b3caE
Network	RHC

Source Files

Filename	SHA256
RobinFlowStreams.sol	756b4850f543996a27456577f1a1f67373360f5f9ebf5d9bcf1cb72ca857f390
RobinFlowFeeCollector.sol	94522188b928998c43ed0bb2d42f815fcc216f8166105dcc6e41fcbbd4921a46

Overview

The RobinFlow system is a two contract suite built around a token vesting and locking product. Users create streams that release a deposited ERC20 token to a recipient over time, with an optional cliff and a linear or tranched unlock schedule. Cancelable streams return the unvested balance to the sender, and transferable streams can be handed to a new recipient. A flat native coin fee is charged on every stream created. RobinFlowStreams.sol is the vesting core, and RobinFlowFeeCollector.sol is the inherited base that handles the fee logic.

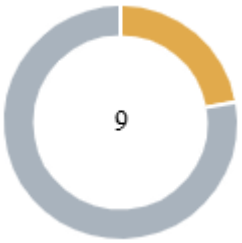
RobinFlowStreams

Holds the full vesting lifecycle creating streams (single or batch), tracking vesting per stream, letting recipients withdraw what's unlocked, and handling cancels and recipient transfers. The contract itself escrows all deposited tokens.

RobinFlowFeeCollector

The shared fee layer every stream creation runs through. It charges a flat native-coin fee, capped by a hard maximum, and lets the owner update the fee or withdraw what's collected. It holds no ERC-20 user funds and has no power over streams.

Findings Breakdown



Critical	0
Medium	2
Minor / Informative	7

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	2	0	0	0
Minor / Informative	7	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Medium	AME	Address Manipulation Exploit	Unresolved
Medium	MC	Missing Check	Unresolved
Minor / Informative	SEW	Stuck Excess Withdrawal	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Unresolved
Minor / Informative	EFK	Extra Fee Kept	Unresolved
Minor / Informative	STI	Stream Transfer Integrity	Unresolved
Minor / Informative	UBS	Unbounded Batch Size	Unresolved
Minor / Informative	VABC	Vesting Accrues Before Cliff	Unresolved
Minor / Informative	L19	Stable Compiler Version	Unresolved

AME - Address Manipulation Exploit

Criticality	Medium
Location	RobinFlowStreams.sol#L134,213
Status	Unresolved

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

As an example, the following two issues may arise.

The cancel path pays the recipient and then the sender within the same transaction, and the two transfers are coupled, so a revert on either one reverts the entire call. If the supplied token is a blocklist-style token such as `USDC` or `USDT`, and the recipient is frozen while still owed a nonzero amount, the transfer to the recipient reverts and the whole `cancel` reverts with it. The sender is then left with no way to reclaim the unvested balance, since `cancel` is the only function that returns it, and the recipient cannot use `withdraw` either, because it sends to the same frozen address. The same outcome occurs with any recipient that causes the payout to revert, including a contract with a reverting token-receipt hook.

```
if (recipientAmount > 0) IERC20(s.token).safeTransfer(s.recipient,
recipientAmount);
if (senderAmount > 0) IERC20(s.token).safeTransfer(s.sender,
senderAmount);
```

The amount actually received is measured once at creation and stored as `deposited`, and the balance is never re-read afterwards, while every stream that uses the same token draws from one shared balance held by the contract. If the supplied token can reduce holder balances after the fact, as elastic tokens do on a downward rebase and as some deflationary tokens do by burning from holders, the contract's real balance falls below the sum of what all its streams are owed, while each stream still shows its original `deposited`. The first account to withdraw is paid in full, and a later account finds the shared balance short and its transfer reverts, even though its stream still shows an untouched balance.

```
  uint256 balBefore = IERC20(p.token).balanceOf(address(this));
    IERC20(p.token).safeTransferFrom(msg.sender, address(this),
p.amount);
    uint256 received = IERC20(p.token).balanceOf(address(this)) -
balBefore;
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

MC - Missing Check

Criticality	Medium
Location	RobinFlowStreams.sol#L122,174,175
Status	Unresolved

Description

The stream parameter validation and the vesting math omit several checks, which let a stream be created with a schedule that does not behave as its stated timeline suggests. The value of `start` is never required to be at or after the current block time, and no minimum spacing is enforced between `start`, `cliff` and `end`. The validation only checks that `end` is after `start` and that `cliff` sits between them, so a stream can be backdated so that most of the amount is already unlocked at creation, can be pushed far into the future, or can have its `start`, `cliff` and `end` placed close together, for example a day or a single second apart, so that a schedule which reads as a long lock releases almost immediately.

```
p.end <= p.start ||
p.cliff < p.start ||
p.cliff > p.end ||
```

The `period` value is never validated. A value of `0` is accepted and treated as per-second unlock, since only values greater than `1` take the tranche branch, so a stream can be created without an intended tranche configuration being enforced.

```
uint32 per = s.period;
    if (per > 1) elapsed = (elapsed / per) * per;
```

Recommendation

The team is advised to validate the schedule parameters according to the intended specification, and to accrue the vested amount from the point the schedule is meant to begin releasing.

SEW - Stuck Excess Withdrawal

Criticality	Minor / Informative
Location	RobinFlowFeeCollector.sol#L56
Status	Unresolved

Description

The `withdrawFees` function withdraws exactly the value tracked in `collectedFees`, rather than the full native balance held by the contract. Any native amount that ends up in the contract outside of this fee accounting is therefore not retrievable, since there is no other function that transfers native value out and the withdrawal is capped to the tracked figure.

```
uint256 amount = collectedFees;
    if (amount == 0) revert NoFees();
    collectedFees = 0; // effects before interaction
    (bool ok, ) = payable(to).call{value: amount}("");
```

Recommendation

The team is advised to provide a way to recover native value that is not part of the tracked fee accounting, so that any amount held by the contract beyond the collected fees can still be retrieved.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	RobinFlowFeeCollector.sol#L35,56
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function setFlatFee(uint256 newFee) external onlyOwner {...}
function withdrawFees(address to) external onlyOwner {...}
function renounceOwnership() public virtual onlyOwner
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

EFK - Extra Fee Kept

Criticality	Minor / Informative
Location	RobinFlowFeeCollector.sol#L49
Status	Unresolved

Description

The function `_collectFeeFor` only checks that the caller sent at least the required fee, but then records the entire payment as collected rather than the fee amount alone. If a caller sends more native coin than required, whether by mistake or because the fee changed just before the transaction landed, the surplus is kept by the contract with no way to retrieve it. As a result, a caller can lose funds by sending more than the required fee.

```
function _collectFeeFor(uint256 count) internal {
    if (msg.value < flatFee * count) revert InsufficientFee();
    collectedFees += msg.value;
    emit FeePaid(msg.sender, msg.value);
}
```

Recommendation

The team should consider only booking the exact fee owed and sending back any extra the caller sent. This protects users from losing money over a simple overpayment.

STI - Stream Transfer Integrity

Criticality	Minor / Informative
Location	RobinFlowStreams.sol#L218
Status	Unresolved

Description

The `transferRecipient` function reassigns a stream to a new recipient but does not keep the recipient records consistent with the stream's state. It does not verify whether the stream has already been canceled, so a canceled stream, which holds no further claimable value, can still be transferred to a new address and emits a `RecipientTransferred` event for a position that carries nothing. In addition, the function appends the stream id to `_incoming[newRecipient]` without removing it from the previous recipient's `_incoming` entry, and a stream transferred back to a former recipient is recorded a second time. As a result, the `incomingStreams` view can return stream ids for an address that is no longer the current recipient, as well as the same id more than once.

```
function transferRecipient(uint256 streamId, address newRecipient)
external {
    Stream storage s = streams[streamId];
    if (msg.sender != s.recipient) revert NotRecipient();
    if (!s.transferable) revert NotTransferable();
    if (newRecipient == address(0)) revert InvalidParams();
    address old = s.recipient;
    s.recipient = newRecipient;
    _incoming[newRecipient].push(streamId);
    emit RecipientTransferred(streamId, old, newRecipient);
}
```

Recommendation

The team is advised to ensure that a stream's recipient records stay consistent with its current state, so that the exposed views reflect only streams that still hold value and list each stream once for the address that currently holds it.

UBS - Unbounded Batch Size

Criticality	Minor / Informative
Location	RobinFlowStreams.sol#L107
Status	Unresolved

Description

There's no cap on how many streams can be created in one batch call. A large enough array will run out of gas and the whole transaction fails. This only affects the person making the call, not anyone else.

```
function createStreamsBatch(StreamParams[] calldata items)
        external
        payable
        nonReentrant
        returns (uint256[] memory ids)
    {
    if (items.length == 0) revert InvalidParams();
    _collectFeeFor(items.length);
    ids = new uint256[](items.length);
    for (uint256 i = 0; i < items.length; i++) {
        ids[i] = _createStream(items[i]);
    }
}
```

Recommendation

The team should consider adding a reasonable max length check on the batch array, and documenting it, so callers know the safe limit ahead of time instead of finding out through a failed transaction.

VABC - Vesting Accrues Before Cliff

Criticality	Minor / Informative
Location	RobinFlowStreams.sol#L175
Status	Unresolved

Description

In the `vestedAmount` he linear portion is calculated using elapsed time from `start`. As a result, the linear portion accrues during the interval between `start` and `cliff`, even though it cannot be claimed during that period. At `cliff`, the recipient receives both the complete `cliffAmount` and all linear vesting accumulated between `start` and `cliff`.

```
function vestedAmount(uint256 streamId, uint40 t) public view returns
(uint256) {
    ...
    uint256 linearPortion = uint256(s.deposited) - s.cliffAmount;
    uint256 elapsed = uint256(t) - s.start;
    uint32 per = s.period;
    if (per > 1) elapsed = (elapsed / per) * per;
    uint256 linear = (linearPortion * elapsed) / (uint256(s.end) -
s.start);
    return uint256(s.cliffAmount) + linear;
}
```

Recommendation

The team is advised to adjust the calculations so that the linear portion begins vesting at the cliff, as well as to calculate both elapsed time and duration relative to cliff.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	RobinFlowStreams.sol#L2
Status	Unresolved

Description

The ^ symbol indicates that any version of Solidity that is compatible with the specifie version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
pragma solidity ^0.8.24;
```

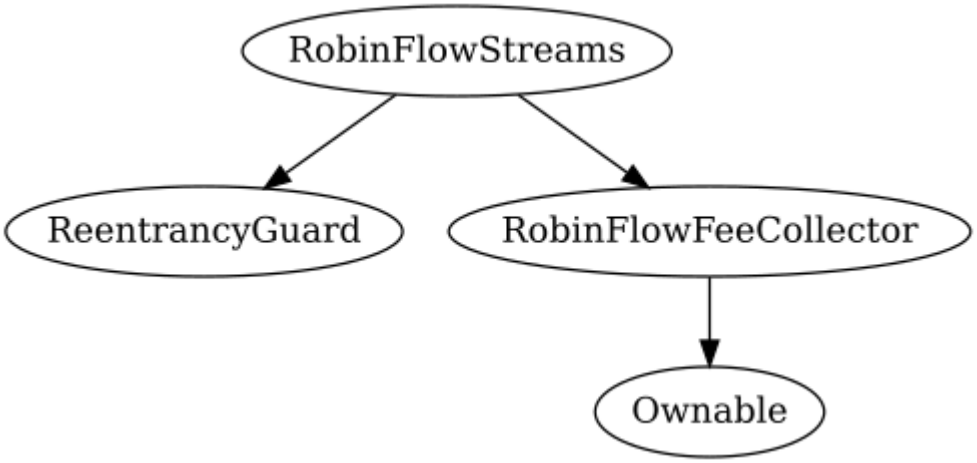
Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

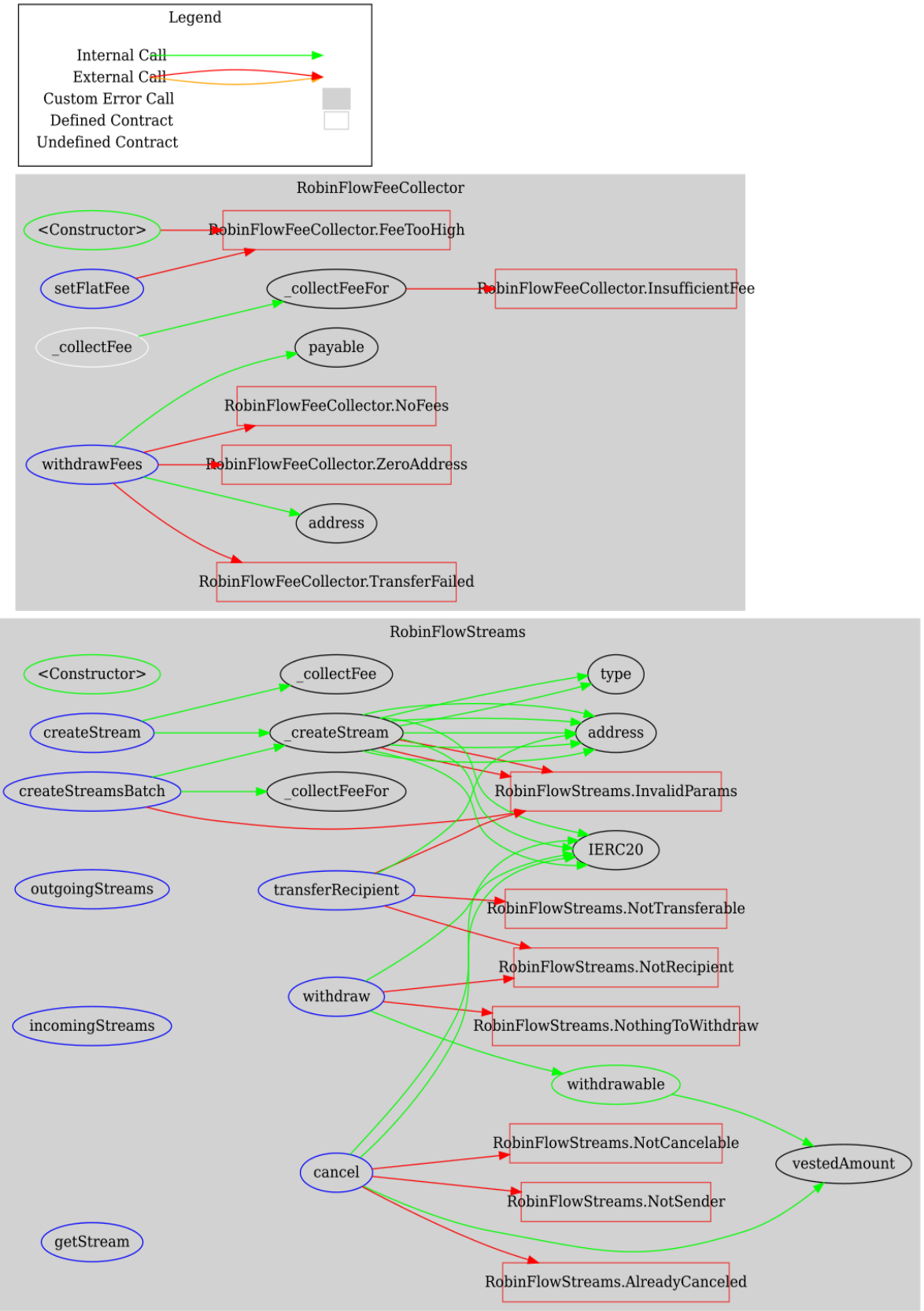
Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
RobinFlowStreams	Implementation	ReentrancyGuard, RobinFlowFeeCollector		
		Public	✓	RobinFlowFeeCollector
	createStream	External	Payable	nonReentrant
	createStreamsBatch	External	Payable	nonReentrant
	_createStream	Private	✓	
	vestedAmount	Public		-
	withdrawable	Public		-
	withdraw	External	✓	nonReentrant
	cancel	External	✓	nonReentrant
	transferRecipient	External	✓	-
	outgoingStreams	External		-
	incomingStreams	External		-
	getStream	External		-

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
RobinFlowFeeCollector
<Constructor>
RobinFlowFeeCollector.FeeTooHigh
setFlatFee
_collectFeeFor
RobinFlowFeeCollector.InsufficientFee
_collectFee
payable
RobinFlowFeeCollector.NoFees
withdrawFees
RobinFlowFeeCollector.ZeroAddress
address
RobinFlowFeeCollector.TransferFailed
RobinFlowStreams
<Constructor>
_collectFee
type
createStream
_createStream
address
createStreamsBatch
_collectFeeFor
RobinFlowStreams.InvalidParams
IERC20
outgoingStreams
transferRecipient
RobinFlowStreams.NotTransferable
RobinFlowStreams.NotRecipient
withdraw
incomingStreams
RobinFlowStreams.NothingToWithdraw
withdrawable
RobinFlowStreams.NotCancelable
vestedAmount
cancel
RobinFlowStreams.NotSender
getStream
RobinFlowStreams.AlreadyCanceled

Summary

RobinFlow contracts implement a token vesting and locking product built around cliffs, linear or tranched release schedules, and cancelable or transferable streams, funded through a flat native coin creation fee. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io